

BCSE497J - Project-I

**CONTEXT-AWARE PERSONAL AI ASSISTANT USING
LARGE LANGUAGE MODELS, DYNAMIC MODEL
ROUTING, MEMORY, AND RETRIEVAL-AUGMENTED
GENERATION (RAG)**

23BAI0111

NISCHAY N

23BAI0105

TEJAS K M

23BCT0227

TENIDHAR REDDY

Under the Supervision of

Prof. Mary Mekala A

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

September 2026

ABSTRACT

Existing AI assistants such as ChatGPT, Claude, and Gemini require users to manually select a model and lose conversational context whenever they switch between them, since no single model is uniformly best across all task types. This project proposes a Context-Aware Personal AI Assistant built on three pillars: Dynamic Model Routing, Persistent Memory, and Retrieval-Augmented Generation (RAG). Following feedback at Review 1, the team studied Perplexity AI's production architecture in depth and found that it already ships all three pillars, which required refining the project's novelty claim. The revised contribution is not a bigger orchestration system, but a more transparent and measurable one: routing generalized beyond search to coding, writing, and multimodal tasks; a published, reproducible routing-scoring function in place of a black-box classifier; formally benchmarked memory instead of casual preference capture; and an open ablation study isolating each component's individual contribution — something not published even by Perplexity itself.

Since Review 1 the team has implemented all four variants on one shared pipeline in which routing, memory and retrieval are independent switches. Model A (routing only) combines an intent classifier (TF-IDF word and character n-grams with a calibrated Linear SVM, trained on 1,350 labelled queries across 5 task categories) with a published accuracy-cost-latency scoring function; it reaches 84.4% held-out accuracy on a leakage-free split, and routing beats an average fixed model by 0.083 points on the 0-1 routing score. Model B adds three-tier persistent memory (turns, session summaries, extracted user facts in SQLite with embedding recall), Model C adds retrieval-augmented generation with numbered, citable passages, and Model D combines all three. An ablation study on small open-weight models served locally through Ollama on an Apple M3 laptop with 8 GB RAM shows memory raising cross-session recall from 0% to 97% (30 questions) and retrieval raising accuracy on a closed-domain question set from 0% to 89% (36 questions), with the combined Model D reaching 97% recall and 89% RAG accuracy. A second, per-task benchmark of 50 automatically graded tasks shows the routed system answering 84% correctly, against 78% for the best single model, with 93.3% routing accuracy on 150 unseen prompts.

Keywords — *Dynamic Model Routing, Large Language Models, Retrieval-Augmented Generation, Persistent Memory, Intent Classification, Ablation Study.*

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	i
1.	INTRODUCTION	1
	1.1 Background	1
	1.2 Motivation	1
	1.3 Scope of the Project	1
2.	PROJECT DESCRIPTION AND GOALS	2
	2.1 Literature Review	2
	2.2 Research Gap	2
	2.3 Objectives	3
	2.4 Problem Statement	3
	2.5 Project Plan	3
3.	TECHNICAL SPECIFICATION	5
	3.1 Requirements	5
	3.2 Feasibility Study	5
	3.3 System Specification	6
4.	DESIGN APPROACH AND DETAILS	7
	4.1 System Architecture	7
	4.2 Design	8
5.	IMPLEMENTATION AND RESULTS	11
	5.1 Implementation	11
	5.2 Experimental Setup	11
	5.3 Results	12
	5.4 Prototype Demonstration	16
	5.5 Discussion	19
	5.6 Work Remaining Before the Final Review	21
6.	REFERENCES	22

1. INTRODUCTION

1.1 Background

Large language models (LLMs) such as GPT, Claude, Gemini, and search-tuned models like Perplexity's Sonar each have distinct strengths — one may excel at coding, another at long-form writing, another at real-time factual lookup. Today, using the best model for each task requires a person to manually judge the task and switch tools, losing conversational context and memory every time they do. A production system, Perplexity AI, already demonstrates that routing, memory, and retrieval can be combined at scale — but its routing logic is undisclosed, its memory is not formally benchmarked, and it is scoped to search-style question answering.

1.2 Motivation

The motivation for this project is twofold. First, no existing academic work combines dynamic model routing, persistent memory, and retrieval-augmented generation into a single evaluated system — each prior paper (RAG, ReAct, Toolformer, MemGPT, LongMem) addresses one piece in isolation. Second, while Perplexity proves this combination is commercially viable, it leaves open exactly the questions a rigorous engineering project can answer: which component contributes how much, whether routing generalizes past search, and whether a routing decision can be made transparent and reproducible rather than a black box.

1.3 Scope of the Project

This project's scope covers the design, implementation, and evaluation of a Context-Aware Personal AI Assistant across four components: (A) dynamic model routing driven by a trained intent classifier and a transparent scoring function; (B) persistent, benchmarked memory; (C) retrieval-augmented generation; and (D) their combination. At this stage all four variants are implemented on a single pipeline, an interactive Streamlit prototype exposes every routing, memory and retrieval decision, and the first full ablation study has been run. Remaining work before the Final Review is to strengthen the evaluation (larger benchmarks, measured rather than prior accuracy figures in the scoring table, and cloud-model runs) rather than to build new components.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Five academic works and one production system were reviewed to establish the state of the art in the three pillars this project builds on. Each academic paper contributes one piece — retrieval grounding, tool use, or memory — in isolation; Perplexity AI is the only known system to combine all three in production, and is therefore treated as the primary related work rather than only an evaluation baseline.

Paper	Objective	Methodology	Pros / Cons	Findings
Lewis et al., RAG (NeurIPS 2020)	Ground LLM outputs in retrieved documents	Dense passage retriever + seq2seq generator	+ Cuts hallucination / – No routing or memory	RAG beats parametric-only models on factual QA
Yao et al., ReAct (ICLR 2023)	Interleave reasoning with tool-use actions	Alternates “thought”/“action” steps in-prompt	+ Better multi-step tasks / – No persistent memory	Reasoning+acting beats reasoning-only or acting-only
Schick et al., Toolformer (NeurIPS 2023)	Teach an LLM when/how to call external tools	Self-supervised fine-tuning on API-call data	+ Model-agnostic tool use / – No multi-LLM routing	Tool use lifts math/QA/lookup performance
Packer et al., MemGPT (2023)	OS-style memory hierarchy for long context	Virtual context paging between memory tiers	+ Long-term memory / – No RAG or routing	Enables coherent chats beyond context limits
Wang et al., LongMem (NeurIPS 2023)	Decouple memory storage from a frozen LLM	Side network retrieves cached past activations	+ Cheap memory scaling / – Single backbone, no routing	Beats standard LLMs on long-context recall
Perplexity AI — production architecture (2025–26)	Search-grounded Q&A combining all 3 pillars	Classifier → router across Sonar/GPT/Claude/Gemini + hybrid RAG	+ Proves orchestration at scale / – Black-box routing, no ablation	All 3 pillars viable in production — but not measured or generalized

2.2 Research Gap

At Review 1, the team's claimed novelty was that “no existing system combines dynamic routing, persistent memory, and RAG.” Following the guide's feedback, the team studied Perplexity AI's published architecture in depth (Section 2.1, row 6) and found this claim no longer holds: Perplexity already ships model-agnostic routing across GPT, Claude, Gemini, and its own Sonar model, cross-session memory, and hybrid lexical-semantic RAG with inline citations.

What remains genuinely open, and forms this project's revised research gap, is: (1) routing generalized across task classes beyond search — coding, long-form writing, and multimodal tasks; (2) a transparent, published routing-scoring function, in contrast to Perplexity's undisclosed classifier weights; (3) a formally structured and benchmarked memory-retrieval system, rather than casual preference capture; and (4) an open, ablatability evaluation that isolates how much each component (routing / memory / RAG) individually contributes to overall system quality — a study neither prior academic work nor Perplexity has published.

2.3 Objectives

1. Train an intent classifier reaching $\geq 90\%$ held-out accuracy across 5 task categories. Status: Partially achieved. 84.4% accuracy (macro F1 0.844) on a group-aware split of 1,350 queries in which paraphrases of test queries never appear in training. The 97.0% reported at Review 2 used a random split that leaks paraphrases; the same random split on the current dataset reports 100.0%, so the leakage-free figure is the one reported.
2. Design and publish a transparent routing-scoring function (Accuracy $\times$ Cost $\times$ Latency) that reproducibly selects a model per task. Status: Achieved. Every decision returns its per-model score breakdown; routing beats the average fixed model by 0.083 points.
3. Implement a persistent, benchmarked memory module (Model B) and measure recall accuracy. Status: Achieved. 96.7% recall on 30 cross-session questions (baseline without memory: 0.0%).
4. Implement a retrieval-augmented generation pipeline (Model C) and measure hallucination reduction against a no-retrieval baseline. Status: Achieved. Accuracy 0.0% $\rightarrow$ 88.9%; hallucination rate 52.8% $\rightarrow$ 8.3%.
5. Conduct a full ablation study across Models A, B, C, and D, and publish the comparative results. Status: First full run complete (Chapter 5); to be repeated with larger benchmarks and cloud models before the Final Review.

2.4 Problem Statement

Users of AI assistants must manually choose a model per task and lose context when switching, while the one production system that automates this (Perplexity AI) does so with an undisclosed routing mechanism, unbenchmarked memory, and no published study isolating which architectural component drives its performance. This project addresses the problem of building and evaluating a context-aware assistant whose routing, memory, and retrieval components are individually measured, transparent, and generalizable beyond search-style tasks.

2.5 Project Plan

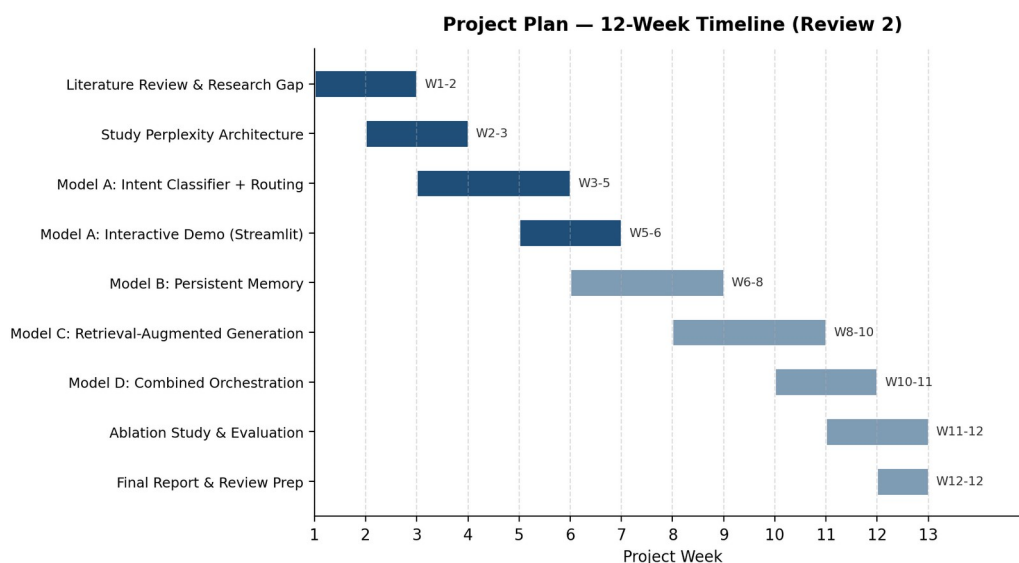


Fig. 1. Project Plan — 12-Week Gantt Chart

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- Classify each incoming query into a task-type intent (Coding / Long-form Writing / Search-Factual / General Reasoning / Multimodal).
- Route classified queries to the best-suited model using a transparent, published scoring function.
- Allow scoring weights (accuracy, cost, latency) to be configured and audited by the user.
- Retrieve relevant context passages to ground responses (Model C).
- Maintain persistent memory of user profile, extracted facts and session summaries across sessions (Model B).
- Generate responses constrained to retrieved context, with numbered citations.
- Provide an interactive interface to run queries and inspect routing decisions live.

3.1.2 Non-Functional

- Performance: classification and routing decisions should complete in well under one second on commodity hardware.
- Scalability: the routing and scoring configuration should extend to additional models or task categories without redesign.
- Reliability: the system should default gracefully (skip generation) if a downstream model API is unavailable.
- Usability: routing decisions and confidence scores should be visible and interpretable, not hidden.
- Maintainability: the intent dataset, scoring table, and generation logic are kept as separate, independently editable modules.
- Transparency: the scoring function and its weights must be inspectable and reproducible by any user.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project uses well-documented, freely available libraries (scikit-learn, pandas, NumPy, Streamlit, SQLite) and runs language models locally through Ollama, so no paid API or GPU is required. The intent classifier trains in seconds on a CPU, and every provider (local or cloud) sits behind one interface.

3.2.2 Economic Feasibility

Development cost is limited to engineering time and, optionally, LLM API usage for the generation step, which can be deferred or run against free-tier quotas during development. No specialized infrastructure investment is required at the current prototype stage.

3.2.3 Social Feasibility

The prototype is a research and learning tool at this stage, with no end-user deployment; ethical considerations around data privacy and model transparency are addressed directly by the project's core contribution — making routing decisions transparent and auditable rather than opaque.

3.3 System Specification

3.3.1 Hardware Specification

- Processor: any modern multi-core CPU; evaluation run on an Apple M3 laptop (no discrete GPU)
- Memory (RAM): 8 GB minimum (small 1–4B-parameter local models); 16 GB for 7–8B models
- Storage: about 10 GB for local model weights; under 500 MB for code, data and results

3.3.2 Software Specification

- Operating System: Windows / macOS / Linux (cross-platform)
- Programming Language: Python 3.10+
- Development Environment: Visual Studio Code
- Libraries and Frameworks: scikit-learn, pandas, NumPy, joblib, Streamlit, matplotlib, pytest; Ollama runtime; optional OpenAI / Anthropic / Gemini APIs
- Database: SQLite for persistent memory; pickled embedding index for retrieval; CSV / JSON datasets

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

Context-Aware Personal AI Assistant: System Architecture (Model D)

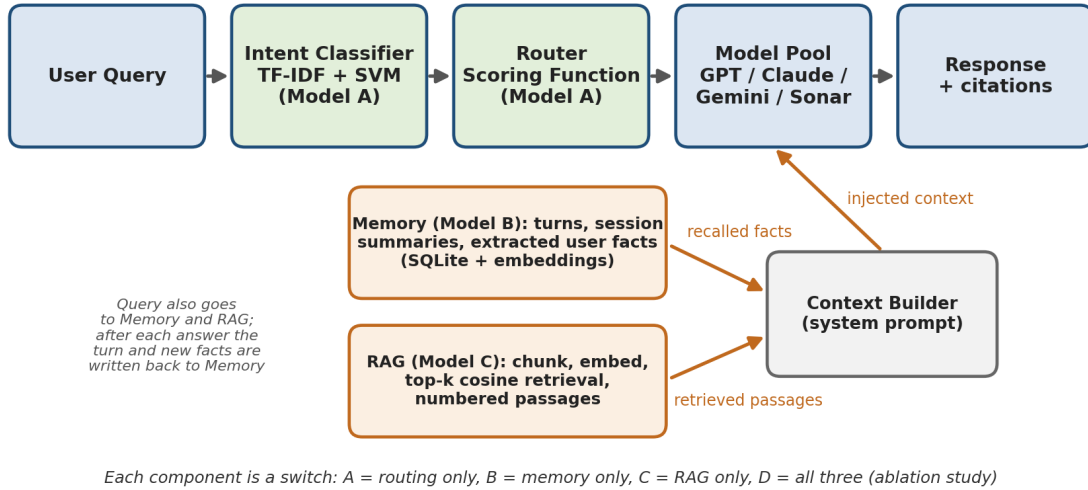


Fig. 2. System Architecture

A user query is tagged by the Intent Classifier and the Router applies the published scoring function, $\text{score} = w_a \cdot \text{Accuracy} + w_c \cdot (1 - \text{Cost}) + w_l \cdot (1 - \text{Latency})$, to select a model from the pool. In parallel the Memory module recalls the most similar stored user facts and session summaries, and the Retriever returns the top-k document passages. The Context Builder injects both into the system prompt, the selected model generates a cited answer, and the turn is written back to memory. Routing, memory and retrieval are independent switches, which is what makes the ablation in Chapter 5 possible.

4.2 Design

4.2.1 Data Flow Diagram

Data Flow Diagram (Level 0): Full System (Model D)

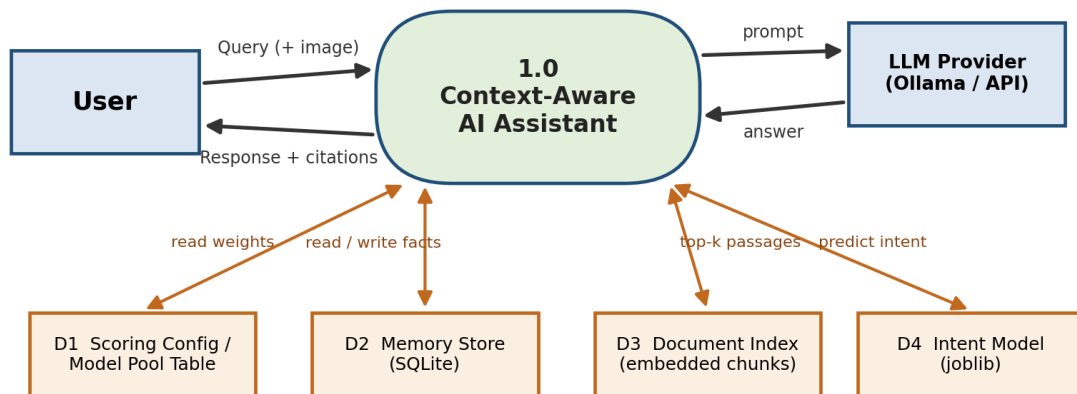


Fig. 3. Data Flow Diagram (Level 0) — Full System (Model D)

4.2.2 Use Case Diagram

Use Case Diagram — Model A: Dynamic Routing Demo

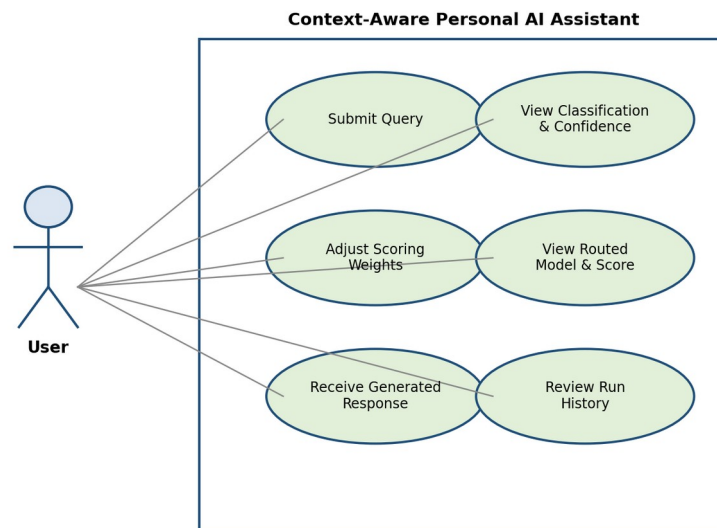


Fig. 4. Use Case Diagram — Prototype

4.2.3 Class Diagram



Fig. 5. Class Diagram — Core Modules

Assistant is the orchestrator; its three boolean flags select the ablation variant. IntentClassifier, Memory and Retriever are only instantiated when their switch is on. Router and Providers are stateless modules: the Router scores the model pool and Providers wraps generation and embedding for Ollama, OpenAI, Anthropic, Gemini or a deterministic mock used by the test-suite.

4.2.4 Sequence Diagram

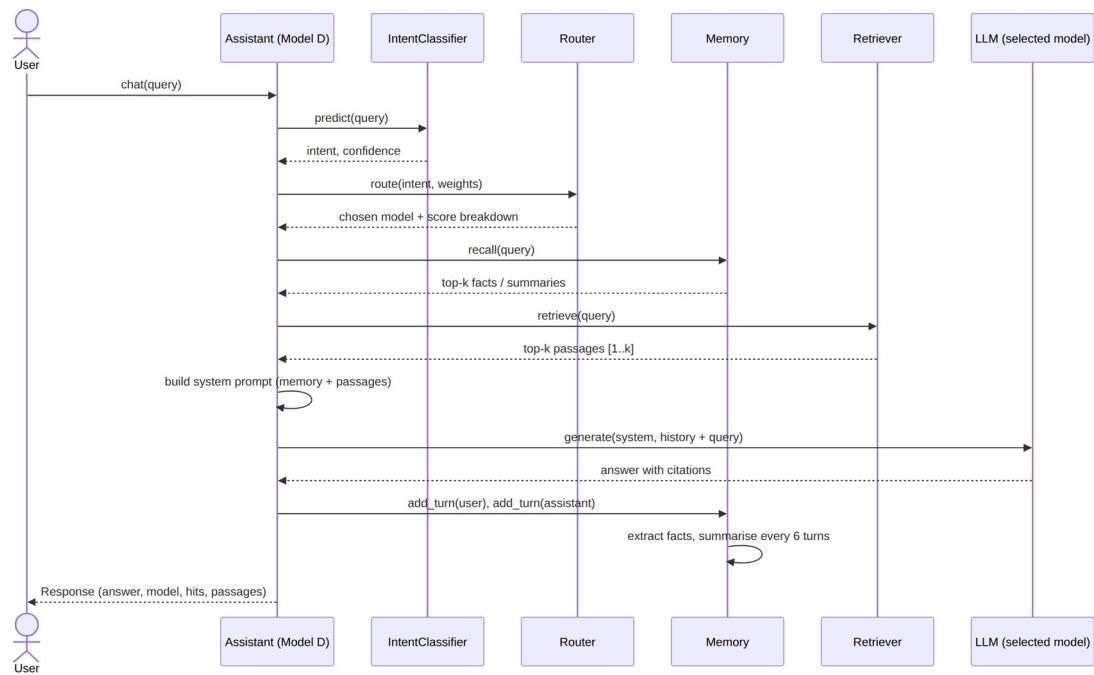


Fig. 6. Sequence Diagram — One Query Through Model D

For each query the assistant classifies intent, routes, recalls memory and retrieves passages before a single generation call; the turn is then stored and new first-person facts are extracted, with a session summary refreshed every six turns.

5. IMPLEMENTATION AND RESULTS

5.1 Implementation

The system is implemented in Python as a single package with one module per component. All variants share the same code path; a variant is defined only by which switches are on.

Module	File	Responsibility
Intent classifier	assistant/intent.py	TF-IDF (word 1-2, char 3-5 grams) + calibrated Linear SVM; 1,350 queries, 5 intents
Router	assistant/router.py	Published weighted score over accuracy, cost and latency; returns full breakdown
Memory	assistant/memory.py	SQLite: turns, session summaries, extracted user facts; cosine top-k recall
RAG	assistant/rag.py	120-word chunks (30 overlap), embeddings, top-4 cosine retrieval, numbered passages
Providers	assistant/providers.py	Ollama / OpenAI / Anthropic / Gemini / mock generation and embeddings
Pipeline	assistant/pipeline.py	Assistant(routing, memory, rag) orchestrator; variants baseline, A, B, C, D
Evaluation	scripts/run_ablation.py	Runs every variant on all benchmarks; writes JSON, CSV and figures
Prototype	app.py	Streamlit chat with a live inspector for intent, scores, memory hits and passages

An offline test-suite (10 tests, deterministic mock model) checks classifier accuracy, routing determinism, weight sensitivity, fact extraction, cross-session recall, retrieval and the end-to-end Model D path.

5.2 Experimental Setup

To keep the study free and reproducible, each role in the model pool is played by a small open-weight model served locally by Ollama. The roles keep the names used in the design so the routing logic is unchanged; swapping in the commercial APIs is a configuration change.

Pool role	Local model used
GPT (coding)	qwen2.5-coder:3b
Claude (writing, reasoning)	llama3.2:3b
Gemini (multimodal)	moondream
Sonar (search)	llama3.2:1b
Embeddings	nomic-embed-text

Four benchmarks were used:

- **Routing:** intent accuracy on a group-aware held-out split (270 test queries, paraphrases of a test query never in training) and routing gain against each fixed model.
- **Memory:** a simulated user states 19+ personal facts across four sessions; a fresh session then asks 30 recall questions.
- **RAG:** 36 questions about a fictional company (six documents) whose facts no model can know from pre-training, so correct answers require retrieval. An answer is a hallucination when it is neither correct nor an explicit abstention.
- **Per-task:** 50 tasks (10 per task type), each phrased three ways, answered by every model and graded automatically (Section 5.3.4).

Non-routing variants always use the GPT-role model, so the baseline differs from Model A only by routing.

5.3 Results

5.3.1 Intent Classification

Intent	Precision	Recall	F1
Coding	1.00	0.72	0.84
Multimodal	0.89	1.00	0.94
Reasoning	0.70	0.89	0.78
Search	0.81	0.72	0.76
Writing	0.91	0.89	0.90
Overall (accuracy / macro F1)		0.844	0.844

Held-out accuracy is 84.4% on 270 unseen queries. A plain random split of the same data reports 100.0% because paraphrases of test queries leak into training; the Review 2 figure of 97.0% was affected by the same leakage and is superseded by the grouped figure.

5.3.2 Routing Gain

Always using	Routed score minus fixed score
GPT	+0.095
Claude	+0.103
Gemini	+0.060
Sonar	+0.075
Mean over fixed models	+0.083

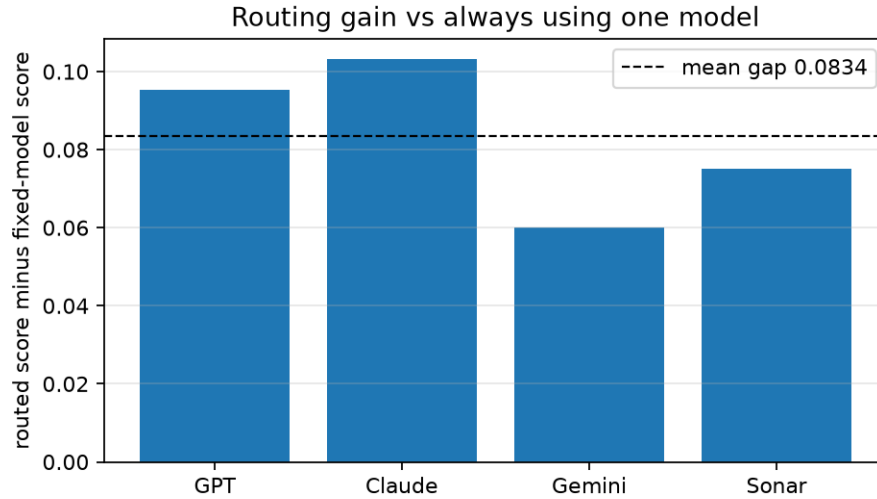


Fig. 7. Routing Gain Over Each Fixed Model

Routing never scores below any fixed model on this measure because it maximises the same score per query. The size of the gain depends on the accuracy, cost and latency priors in the model-pool table, so it is a property of the scoring function rather than of answer quality; Section 5.3.4 measures answer quality directly.

5.3.3 Ablation Study

Variant	Route	Mem	RAG	Memory recall	RAG accuracy	Hallucination	Citations
Baseline	–	–	–	0.0%	0.0%	52.8%	0.0%
Model A	✓	–	–	0.0%	0.0%	13.9%	0.0%
Model B	–	✓	–	96.7%	0.0%	91.7%	0.0%
Model C	–	–	✓	0.0%	88.9%	8.3%	13.9%
Model D	✓	✓	✓	96.7%	88.9%	5.6%	0.0%

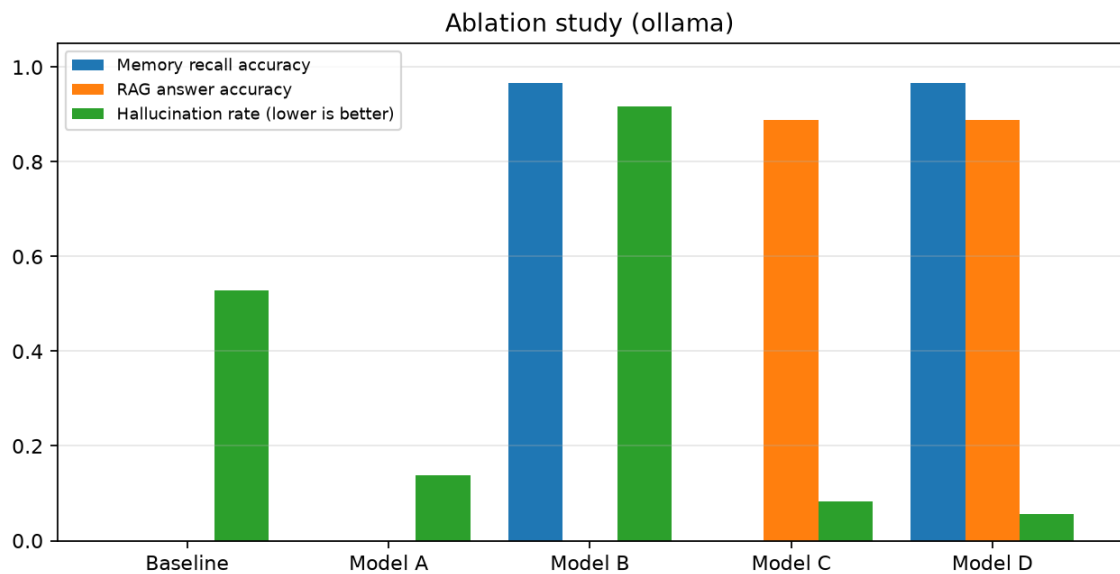


Fig. 8. Ablation Results Across Variants

5.3.4 Per-Task Benchmark: Routing and Answer Accuracy by Task Type

To test whether each kind of task reaches the right model and whether the answer is actually correct, a second benchmark of 50 tasks (10 per intent) was built. Every task is written three ways (direct, casual and verbose), giving 150 prompts for routing. The direct form of every task was answered by every model in the pool at temperature 0 and graded automatically: generated code is executed against unit tests, reasoning answers must match the exact number, factual answers must contain the fact, writing must satisfy explicit constraints (line counts, word limits, required words, valid JSON keys), and image questions use generated test images. Because routing only decides which model answers, the routed system's result on a task is the graded answer of the model it selects.

Prompt style	Intent accuracy
Direct	100.0%
Casual	82.0%
Verbose	98.0%
All 150 prompts (text only)	93.3%
All 150 prompts (image attached => multimodal)	96.7%
Same route for all 3 phrasings of a task	90.0%

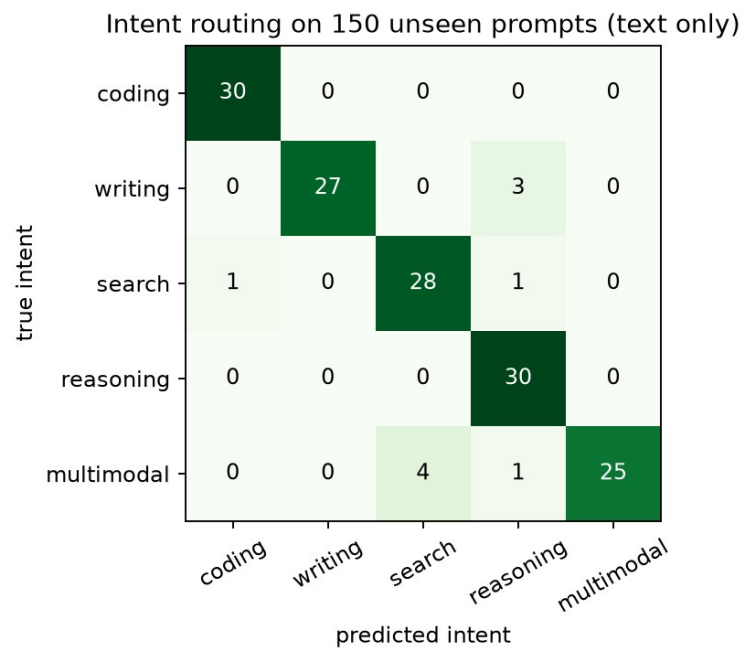


Fig. 9. Intent Confusion Matrix on 150 Unseen Prompts

Model	Coding	Writing	Search	Reason.	Multim.	Overall	Latency
GPT	100%	100%	100%	90%	0%	78%	4.6s
Claude	90%	90%	100%	90%	0%	74%	3.9s
Gemini	80%	60%	60%	0%	50%	50%	3.3s
Sonar	70%	90%	90%	80%	0%	66%	3.2s
Routed system	100%	90%	90%	90%	50%	84%	4.3s

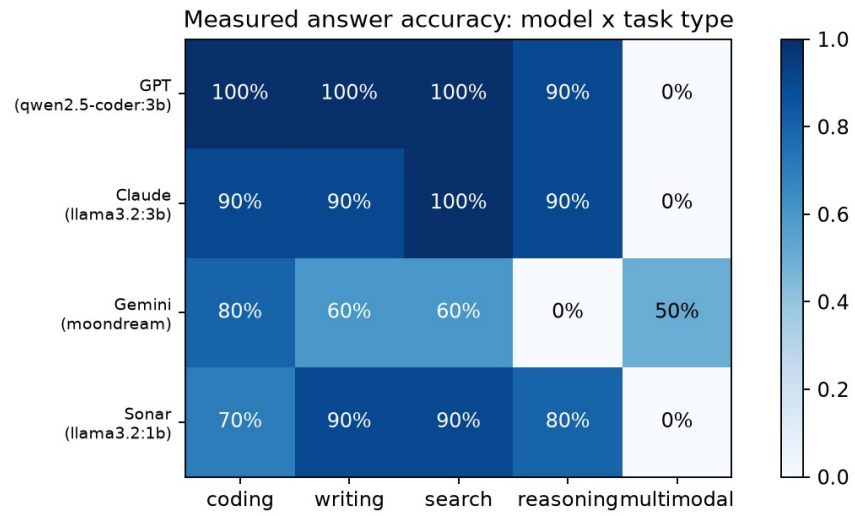


Fig. 10. Measured Answer Accuracy, Model $\times$ Task Type

System	Tasks correct
Routed (classifier + router)	84.0%
Router with true intent	84.0%
Router with measured priors	82.0%
Best model per intent (hindsight)	88.0%
Any model correct (upper bound)	88.0%
Always GPT	78.0%
Always Claude	74.0%
Always Gemini	50.0%
Always Sonar	66.0%

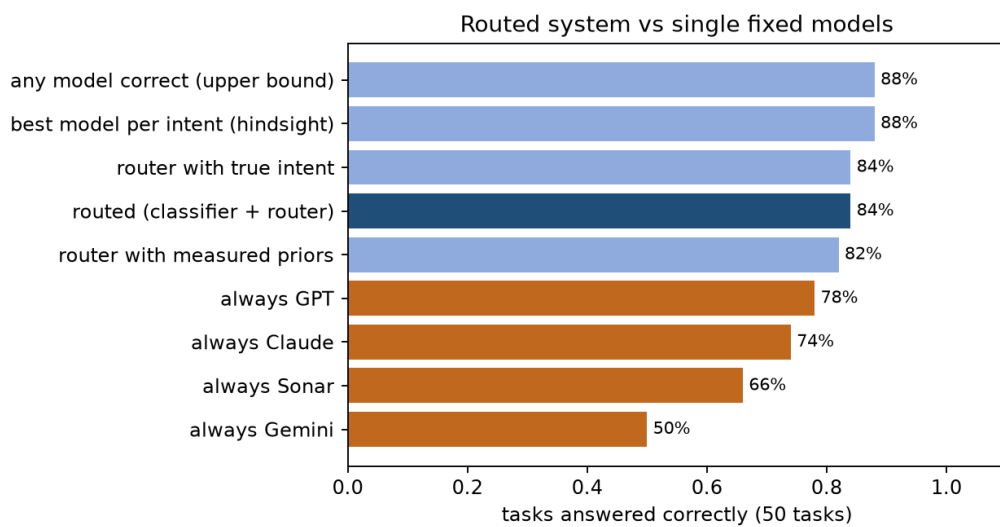


Fig. 11. Routed System vs Single Fixed Models

Task type	Model chosen with prior estimates	Model chosen with measured accuracy	Best in hindsight
Coding	GPT	GPT	GPT
Writing	Claude	Sonar	GPT
Search	Sonar	Sonar	Claude
Reasoning	Claude	Sonar	Claude
Multimodal	Gemini	Gemini	Gemini

- **Routing beats every single model.** The routed system answered 84% of the 50 tasks correctly, against 78% for the best fixed model (GPT role), 74% (Claude role), 66% (Sonar role) and 50% (Gemini role). The best possible choice of model per task type, known only in hindsight, reaches 88%, so routing captures most of the available gain.
- **Classification is near perfect on clear prompts and weakest on casual ones.** Intent accuracy is 100% on direct prompts, 98% on verbose prompts and 82% on casual, abbreviated prompts (for example “how many continents are there” was routed as reasoning). Across 150 prompts it is 93.3%, rising to 96.7% when an attached image forces the multimodal route.
- **Misroutes were cheap in this pool.** All five misroutes sent a writing or search prompt to a strong text model, which still answered correctly, so accuracy with predicted intent equals accuracy with the true intent (84%).
- **Only the vision model can read images.** The three text models rejected image input outright, so multimodal tasks depend entirely on the Gemini-role model (moondream), which read 5 of 10 test images correctly. The same model scored 0% on reasoning, which is exactly the kind of mismatch routing avoids.
- **Measured priors trade accuracy for speed.** Re-routing with the measured accuracies (and measured latency) sends writing and reasoning to the faster 1B Sonar-role model and scores 82%, two tasks below the prior-based router, while cutting average latency. Which is better depends on the accuracy weight, which the user controls.

5.4 Prototype Demonstration

The Streamlit prototype exposes every decision the pipeline makes: the predicted intent and its probabilities, the score of every model under the current weights, the memory items recalled and the passages retrieved, and the exact system prompt sent to the model. The screenshots below were captured from the running prototype on the local Ollama models.

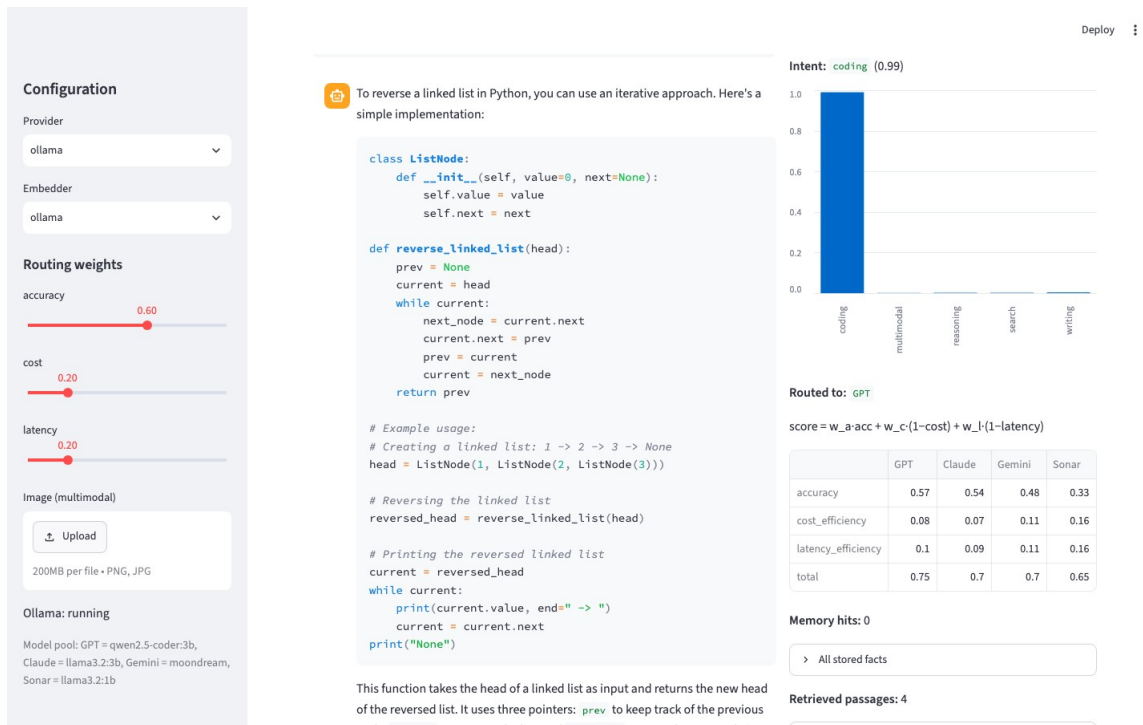


Fig. 12. Coding query (intent confidence 0.99) routed to the GPT-role model, with the per-model score breakdown

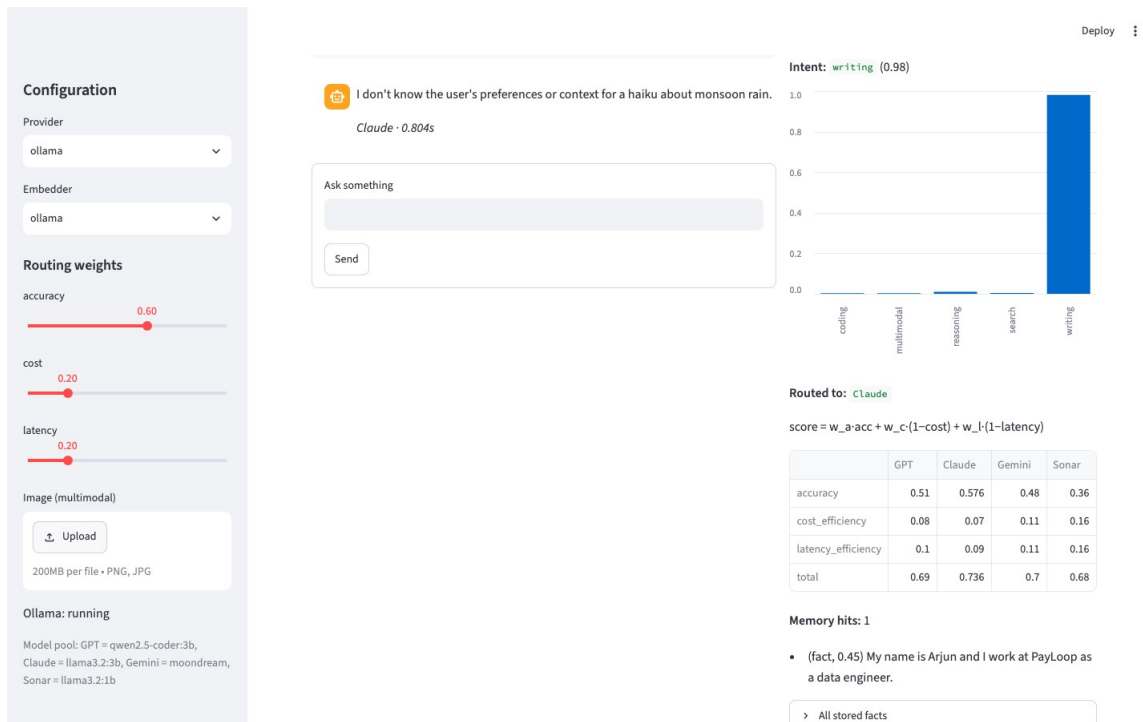


Fig. 13. Writing query routed to the Claude-role model under the default weights

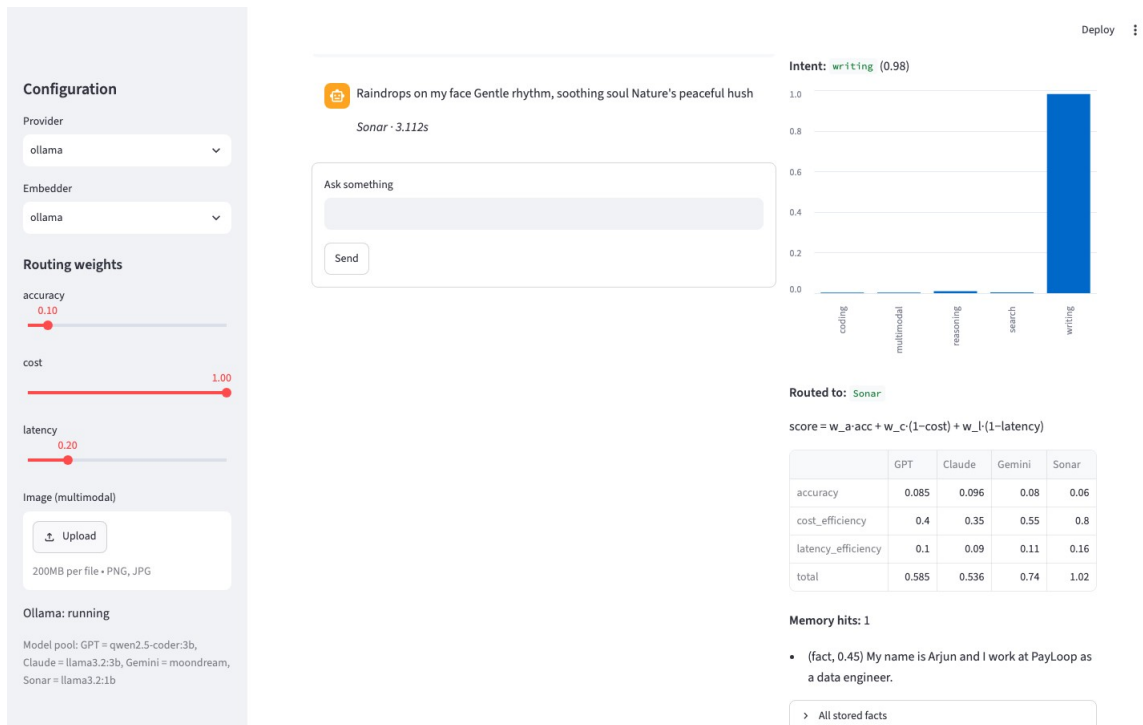


Fig. 14. The same writing query with the cost weight raised to 1.0: the router switches to the cheaper Sonar-role model

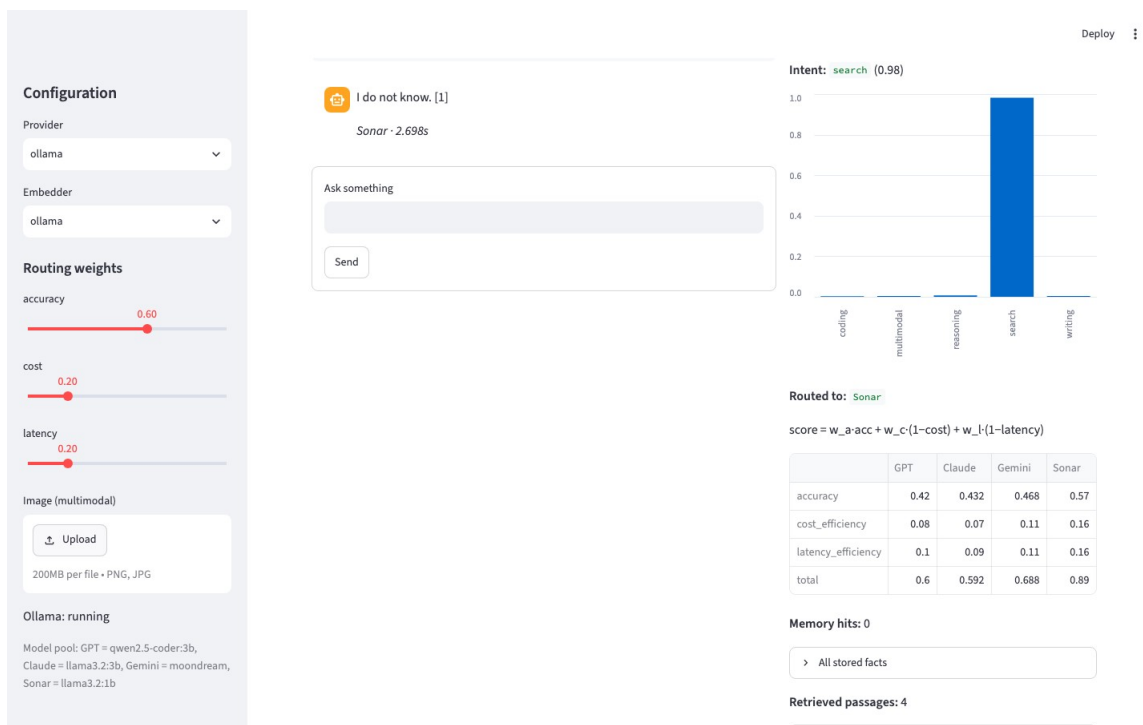


Fig. 15. Factual query routed to the Sonar-role model; the strict grounding prompt produces an abstention (see Section 5.5)

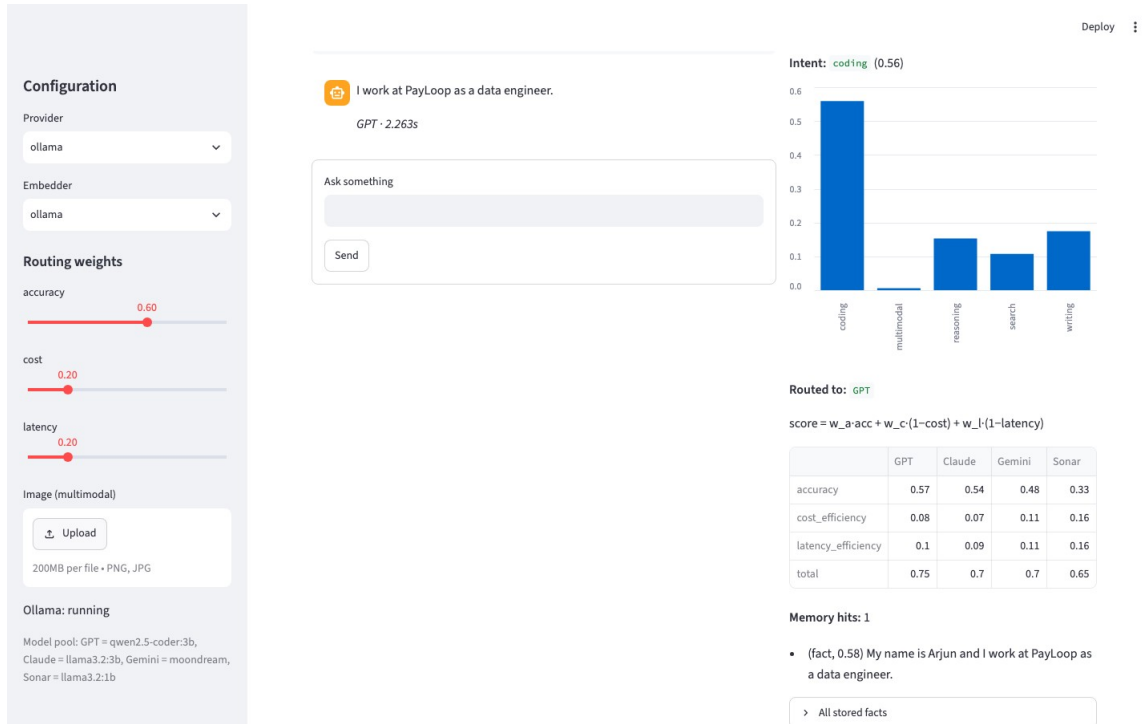


Fig. 16. A fact stated in an earlier session recalled in a new session

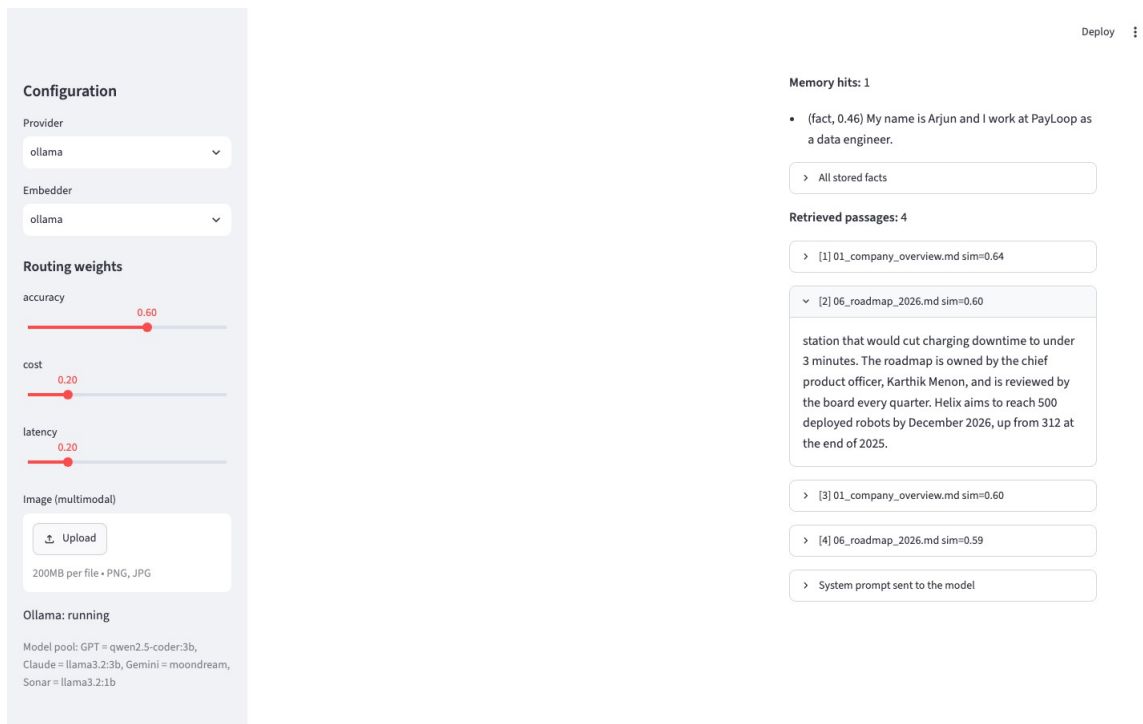


Fig. 17. Retrieved, numbered passages with similarity scores for a closed-domain question

5.5 Discussion

- **Memory is the only source of cross-session recall.** Variants without memory score 0.0% on recall; adding memory lifts it to 96.7% (Model B) and 96.7% (Model D).
- **Retrieval drives factual accuracy and cuts hallucination.** Without retrieval accuracy is 0.0% with a hallucination rate of 52.8%; with retrieval accuracy is 88.9% (Model C) and hallucination falls to 8.3%.

- **Routing changes how a model fails.** With no retrieval every variant scores 0% on the closed-domain questions, but the routed Model A abstains (86.1%) far more often than the fixed-model baseline, cutting hallucination from 52.8% to 13.9%. The search-role model it selects is more willing to say it does not know.
- **Memory without retrieval increases hallucination on unknown facts** (91.7% for Model B). Answers to earlier questions stay in the conversation history, and the model continues in the same confident style instead of abstaining. Memory is valuable for user facts but needs retrieval, or a stricter abstention prompt, beside it.
- **The full system keeps both gains.** Model D matches Model B on recall (96.7%) and Model C on RAG accuracy (88.9%) while lowering hallucination further (8.3% to 5.6%).
- **Citations are rarely emitted** (13.9% for Model C, 0.0% for Model D) even though answers are grounded: the 1-3B local models often ignore the instruction to cite [n]. This is expected to improve with larger or cloud models.
- **Routing** affects which model answers, not what context it sees, so its gain is measured separately in Section 5.3.2.

5.5.1 Positioning Against Perplexity AI

Perplexity AI is the closest production system. The table summarises where this project differs; it is not a head-to-head benchmark, since Perplexity uses live web search and far larger proprietary models.

Aspect	Perplexity AI	This project
Routing rule	Undisclosed classifier	Published weighted formula; weights adjustable by the user
Explanation of each decision	Not shown	Intent probabilities and every model's score, shown per query
Task scope	Search-style question answering	Coding, writing, reasoning, search and images, routed separately
Memory	Personalisation, not publicly benchmarked	Three-tier memory with a 30-question cross-session recall benchmark
Evidence of each component's value	No published ablation	Five-build ablation and a 50-task benchmark with every answer logged
Retrieval source	Live web	The user's own document collection (web search is future work)
Models and cost	Large commercial models, paid	Any provider; evaluated fully offline on free local models

Where Perplexity is stronger: live web retrieval, model scale and production reliability. Where this project contributes: it makes the routing decision inspectable and quantifies what routing, memory and retrieval each add, which no production system publishes.

- **Over-grounding in the full system.** Screenshots of Model D (Figs. 13 and 15) show a side effect of the strict grounding instruction: for general questions such as “What is the capital of Australia?” or a haiku request, retrieval still returns the closest Helix passages, and the prompt “answer only from the retrieved context” makes the model

abstain. The planned fix is to inject passages and the grounding instruction only when retrieval similarity exceeds a threshold. The ablation numbers above were measured with the current behaviour and have not been adjusted.

Limitations: the benchmarks are small (tens of questions); correctness is judged by gold keywords rather than human rating, which occasionally marks a correct answer wrong (for example “No, you do not eat meat” fails the keyword “vegetarian”); the local models are far smaller than the commercial models they stand in for, and the routing priors are estimates. Each is addressed in the plan below.

5.6 Work Remaining Before the Final Review

- Gate retrieval on a similarity threshold so general questions are not forced into the document context.
- Improve intent classification on casual, short prompts (the weakest phrasing style) with more informal training data.
- Enlarge the memory and RAG benchmarks and add an LLM-judge or human rating alongside keyword matching.
- Repeat the ablation with the commercial APIs (GPT, Claude, Gemini) for comparison with the local run.
- Latency and cost measurements per variant; prepare the Scopus conference paper from these results.

6. REFERENCES

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Proc. NeurIPS, 2020, pp. 9459–9474.
- [2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. ICLR, 2023.
- [3] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” in Proc. NeurIPS, 2023.
- [4] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.
- [5] W. Wang et al., “Augmenting Language Models with Long-Term Memory,” in Proc. NeurIPS, 2023.
- [6] ByteByteGo, “How Perplexity Built an AI-Powered Search Engine,” ByteByteGo Newsletter, 2025–2026.
- [7] Ollama, “Get up and running with large language models locally,” <https://ollama.com>, 2026.
- [8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.